

Data Communications

Lecture #9

What we have discussed

- Principles of reliable data transfer

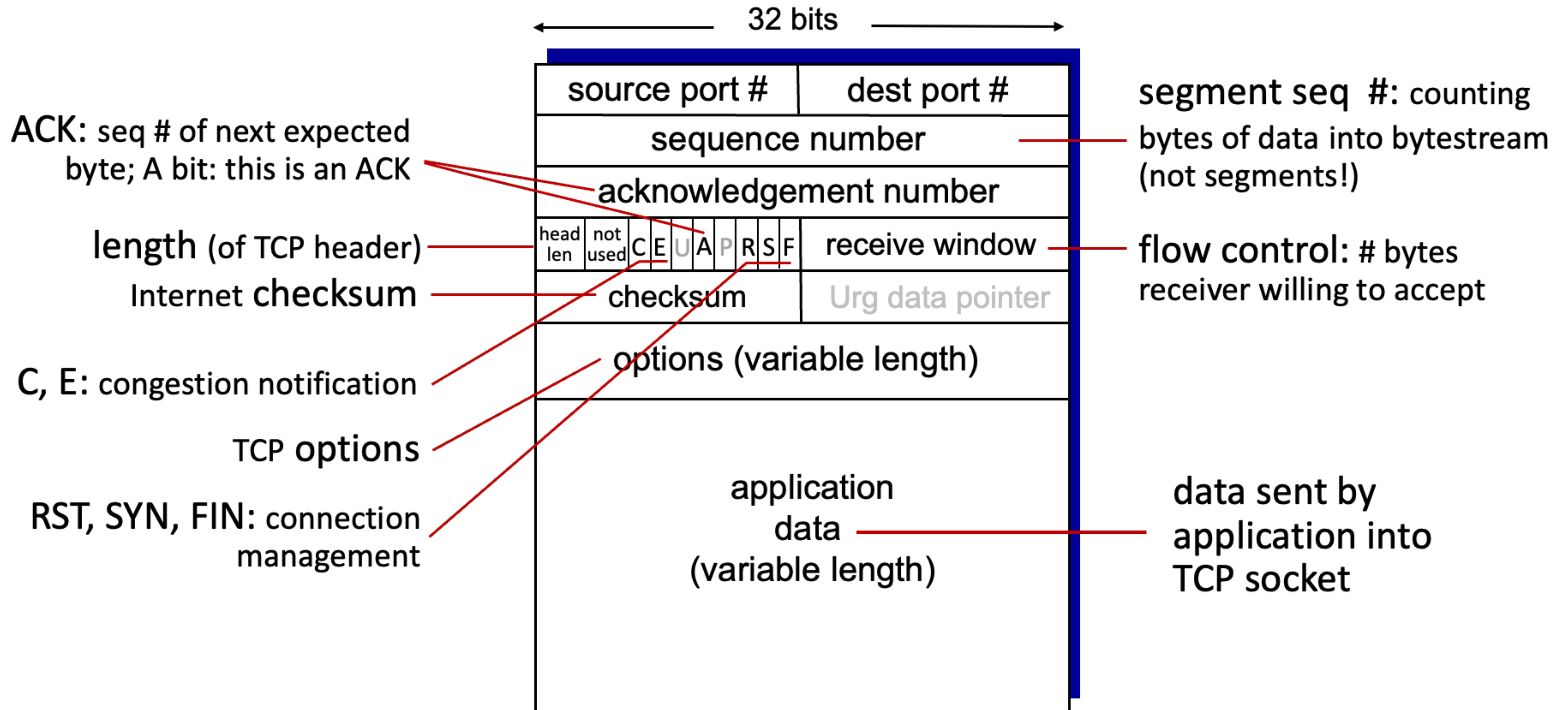
Today...

- Connection-oriented transport: TCP
 - Segment structure
 - Reliable data transfer
 - Flow control
 - Connection management

TCP (RFCs: 793, 1122, 2018, 5681, 7323)

- Point-to-point (end-to-end): one sender, one receiver
- Reliable, in-order byte stream: no “message boundaries”
- Full duplex data
 - Bi-directional data flow in the same connection
 - Maximum segment size (MSS) for each direction
- Cumulative ACKs
- Pipelining: TCP congestion/flow control in-flight traffic using “congestion window”
- Connection-oriented: handshaking (exchange of control messages) initializes sender, receiver state before data exchange
- Flow controlled: sender will not overwhelm receiver

TCP segment structure



TCP sequence numbers, ACKs

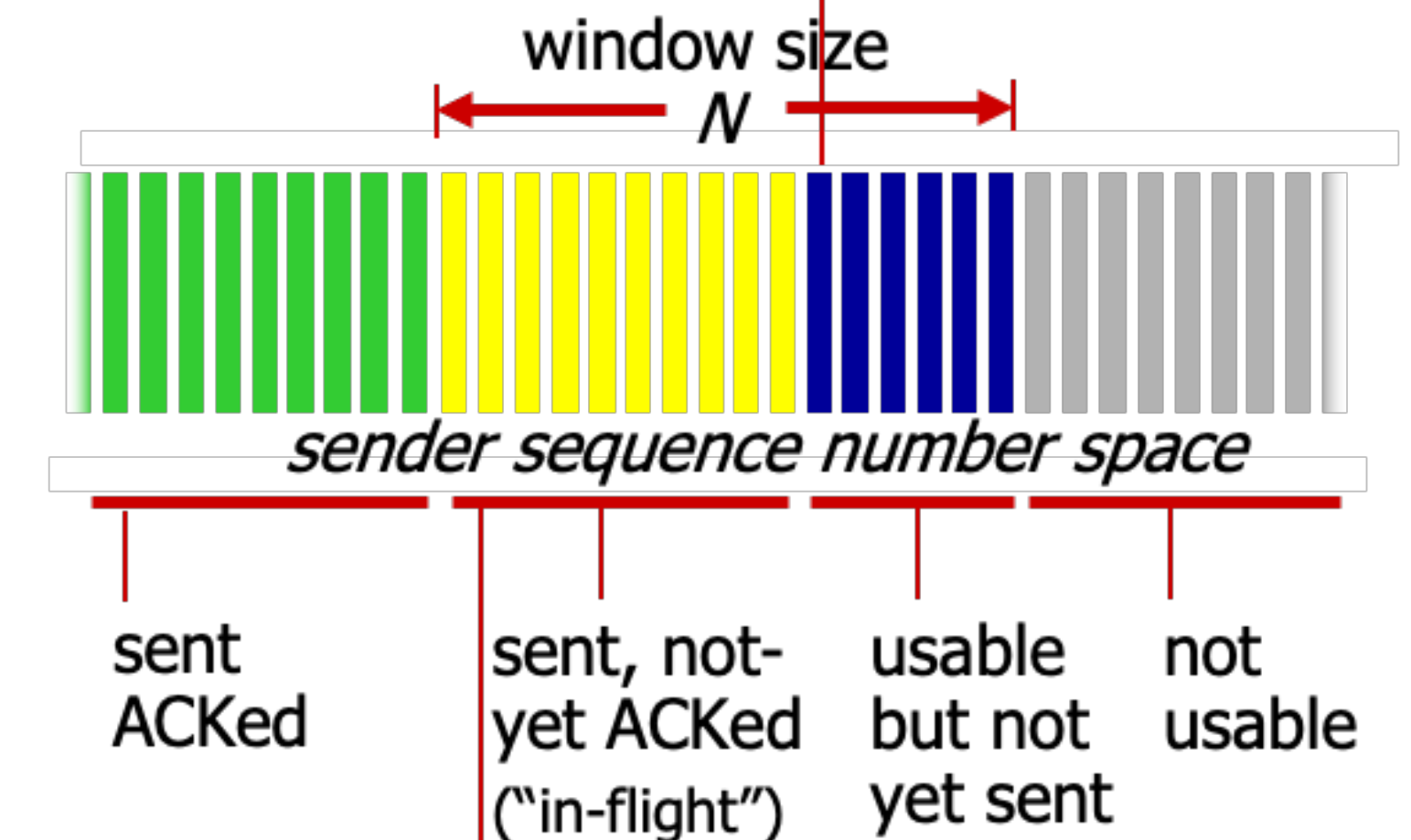
- Sequence numbers
 - Byte stream “number” of the first byte in segment’s data
- Acknowledgements:
 - Seq. # of next byte expected from other side
 - Cumulative ACK

Q) how does receiver handle out-of-order segments?

⇒ TCP spec does not say, -up to implementor

outgoing segment from sender

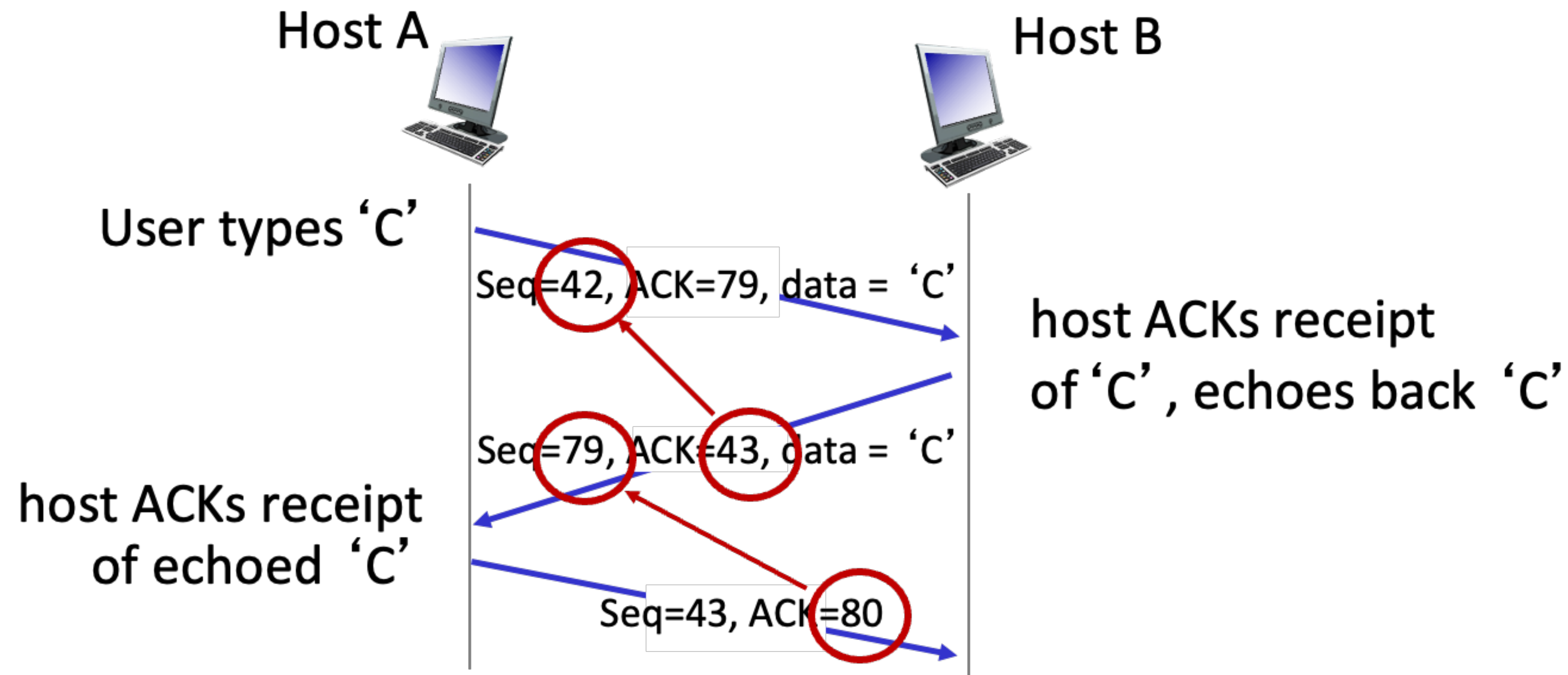
source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer



outgoing segment from receiver

source port #	dest port #
sequence number	
acknowledgement number	
	A
checksum	urg pointer

TCP sequence numbers, ACKs



simple telnet scenario

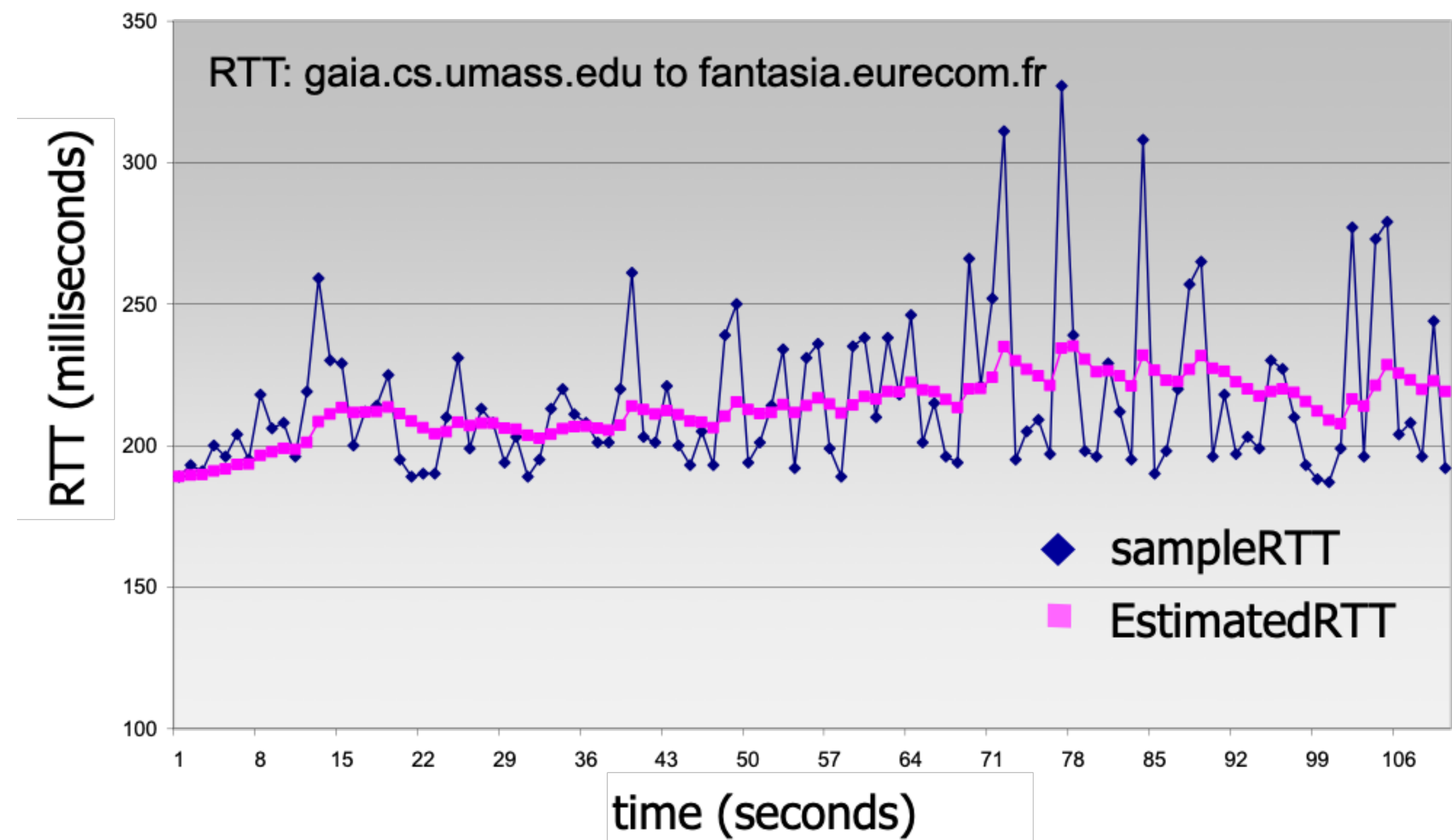
TCP round trip time, timeout

- How to set TCP timeout value?
 - Longer than RTT, but RTT varies!
 - Too short: premature timeout, unnecessary retransmissions
 - Too long: slow reaction to segment loss
- How to estimate RTT?
 - **sampleRTT**: measured time from segment transmission until ACK receipt
 - Ignore retransmissions
 - **SampleRTT** will vary (we want estimated RTT “smoother”)
 - Average several recent measurements, not just current **SampleRTT**

TCP round trip time, timeout

$$\text{EstimatedRTT} = (1 - \alpha) \text{ EstimatedRTT} + \alpha \text{ sampleRTT}$$

- Exponential weighted moving average (EWMA)
- Influence of past sample decrease exponentially fast
- Typical value of α : 0.125



TCP round trip time, timeout

- Timeout interval: **EstimatedRTT** plus “safety margin”
 - Large variation in **EstimatedRTT** (we want a larger safety margin)

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \times \text{DevRTT}$$

↑
estimated RTT

↑
safety margin

- **DevRTT**: EWMA of **SampleRTT** deviation from **EstimatedRTT**

$$\text{DevRTT} = (1 - \beta) \text{DevRTT} + \beta |\text{SampleRTT} - \text{EstimatedRTT}|$$

(typically, $\beta = 0.25$)

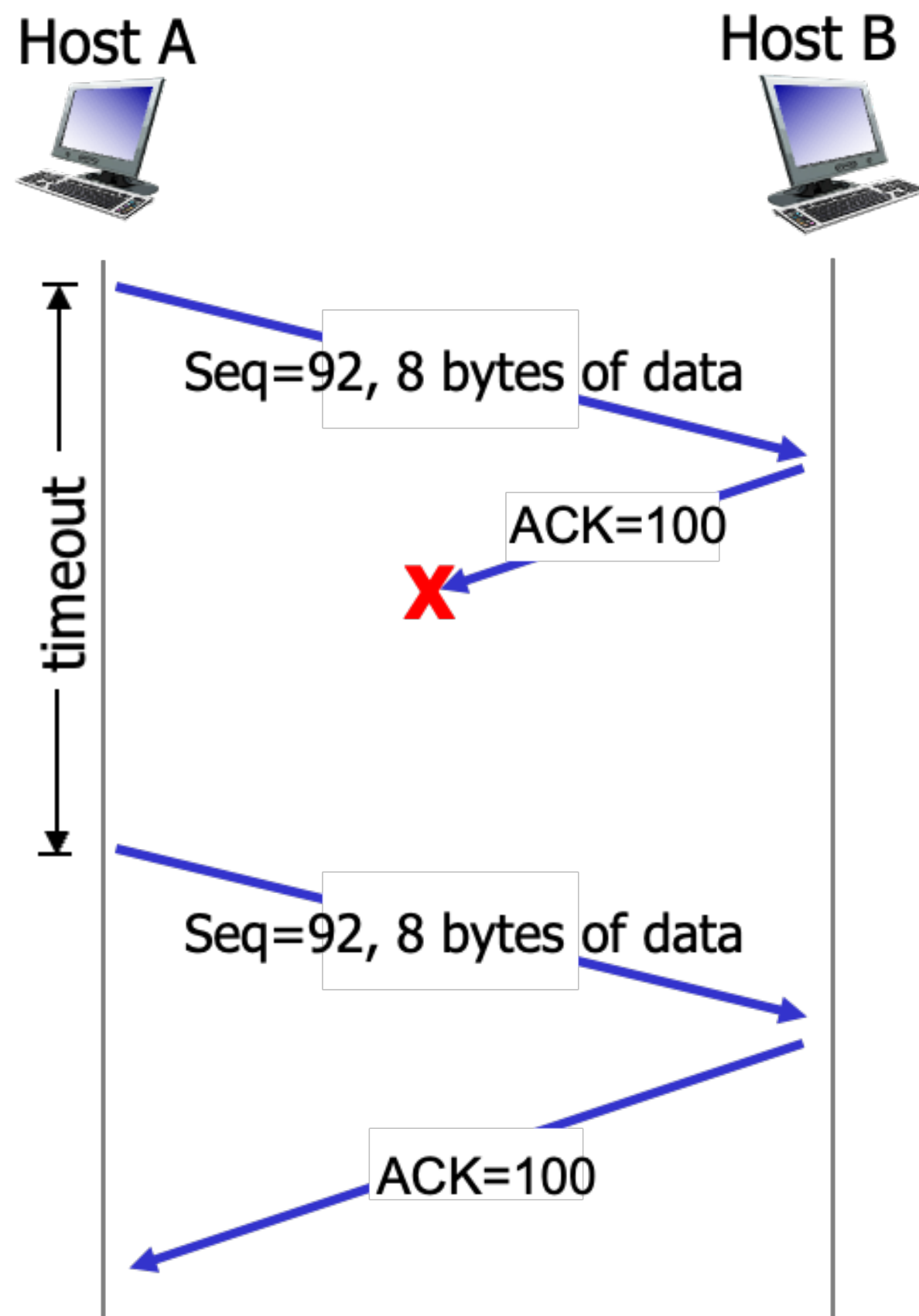
TCP sender (simplified)

- event: data received from application
 - Create segment with seq. #
 - Seq. # is a byte-stream number of the first data byte in segment
 - Start timer if not already running
 - Think of timer as for oldest unACKed segment
 - Expiration interval: **TimeoutInterval**
- event: timeout
 - Retransmit segment that caused timeout
 - Restart timer
- event: ACK received
 - If ACK acknowledges previously unACKed segments
 - Update what is known to be ACKed
 - Start timer if there are still unACKed segments

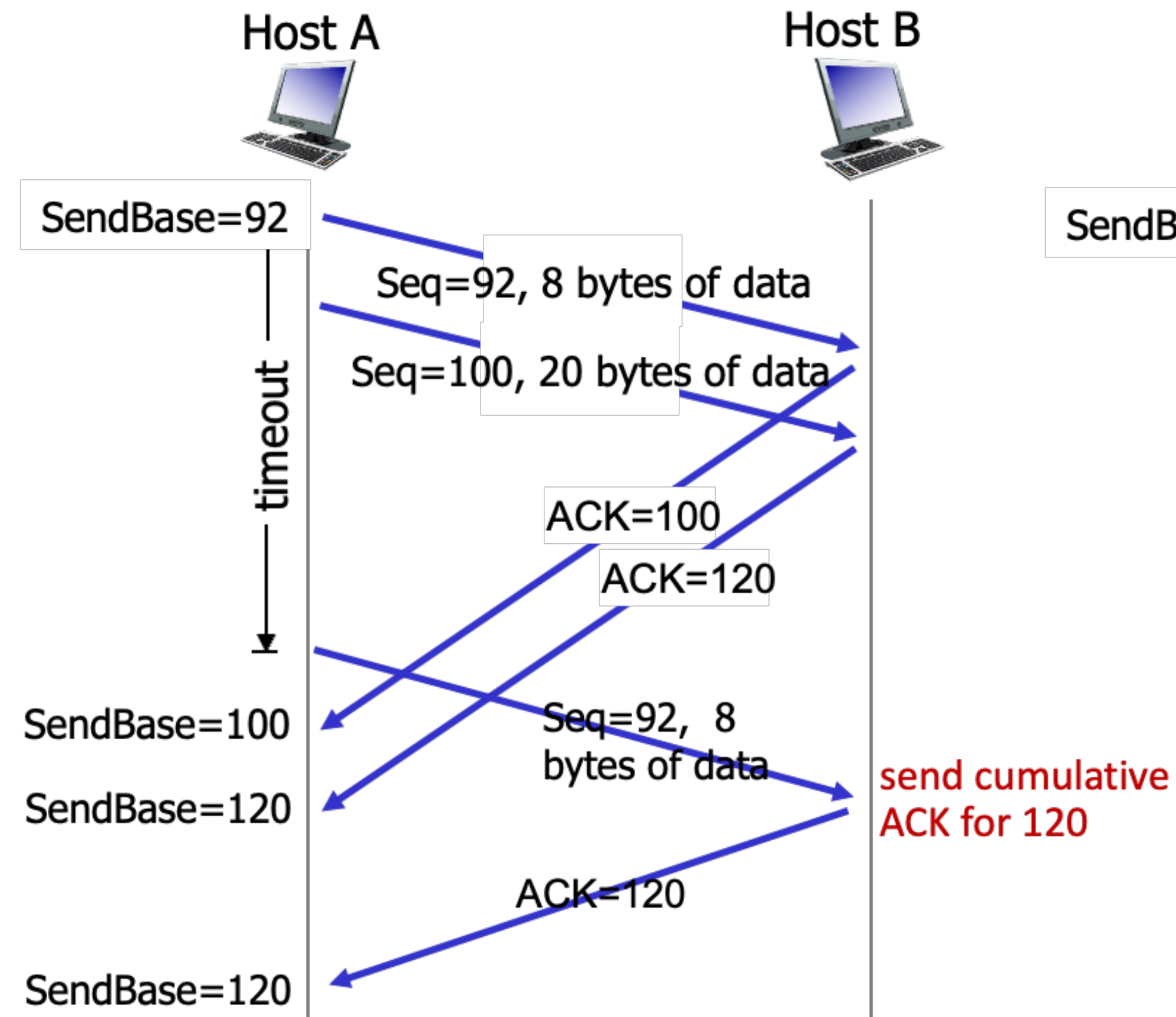
TCP receiver: ACK generation (RFC 5681)

<i>Event at receiver</i>	<i>TCP receiver action</i>
arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK
arrival of in-order segment with expected seq #. One other segment has ACK pending	immediately send single cumulative ACK, ACKing both in-order segments
arrival of out-of-order segment higher-than-expect seq. # . Gap detected	immediately send <i>duplicate ACK</i> , indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediate send ACK, provided that segment starts at lower end of gap

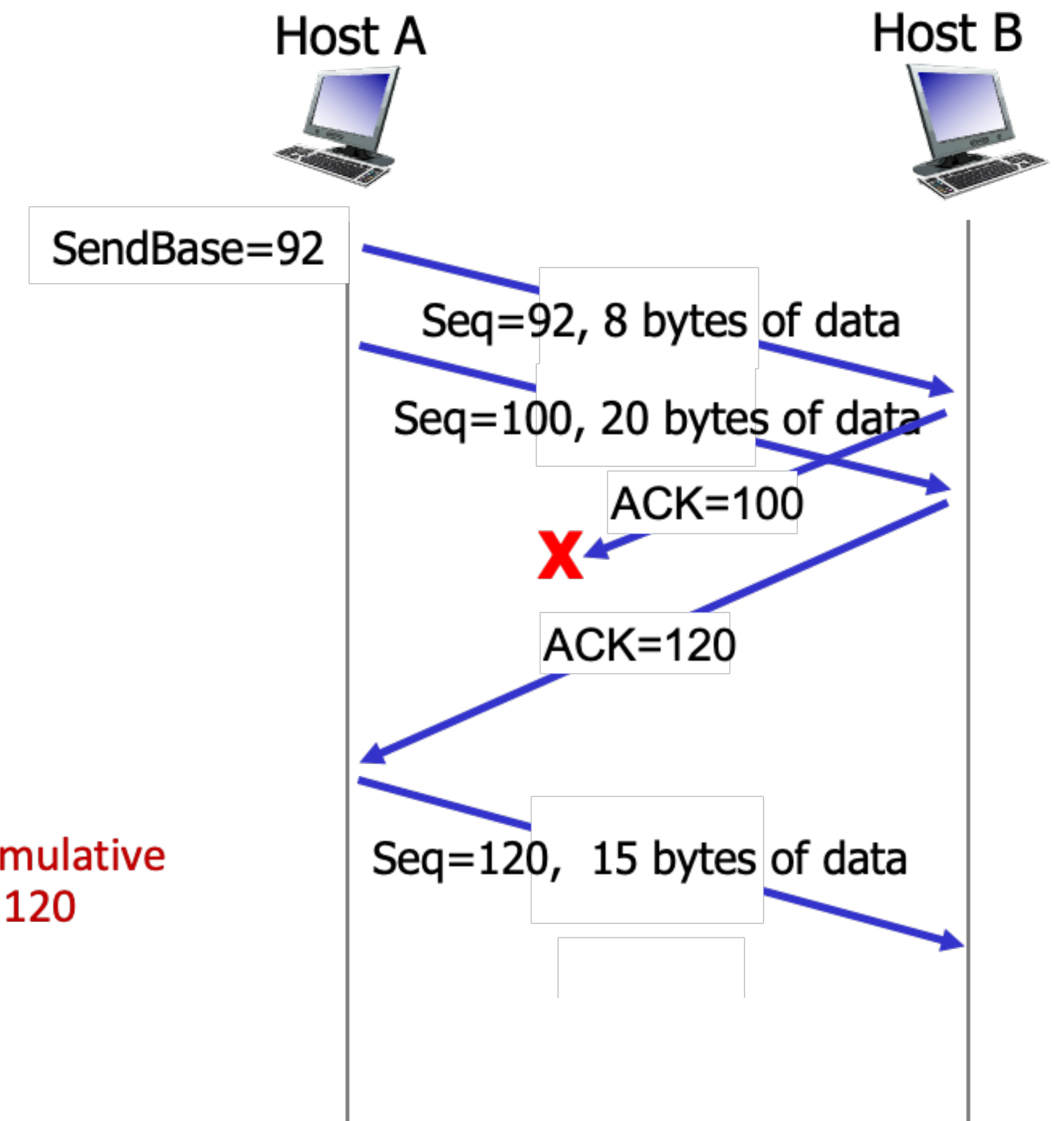
TCP retransmission scenarios



Lost ACK scenario



Premature timeout



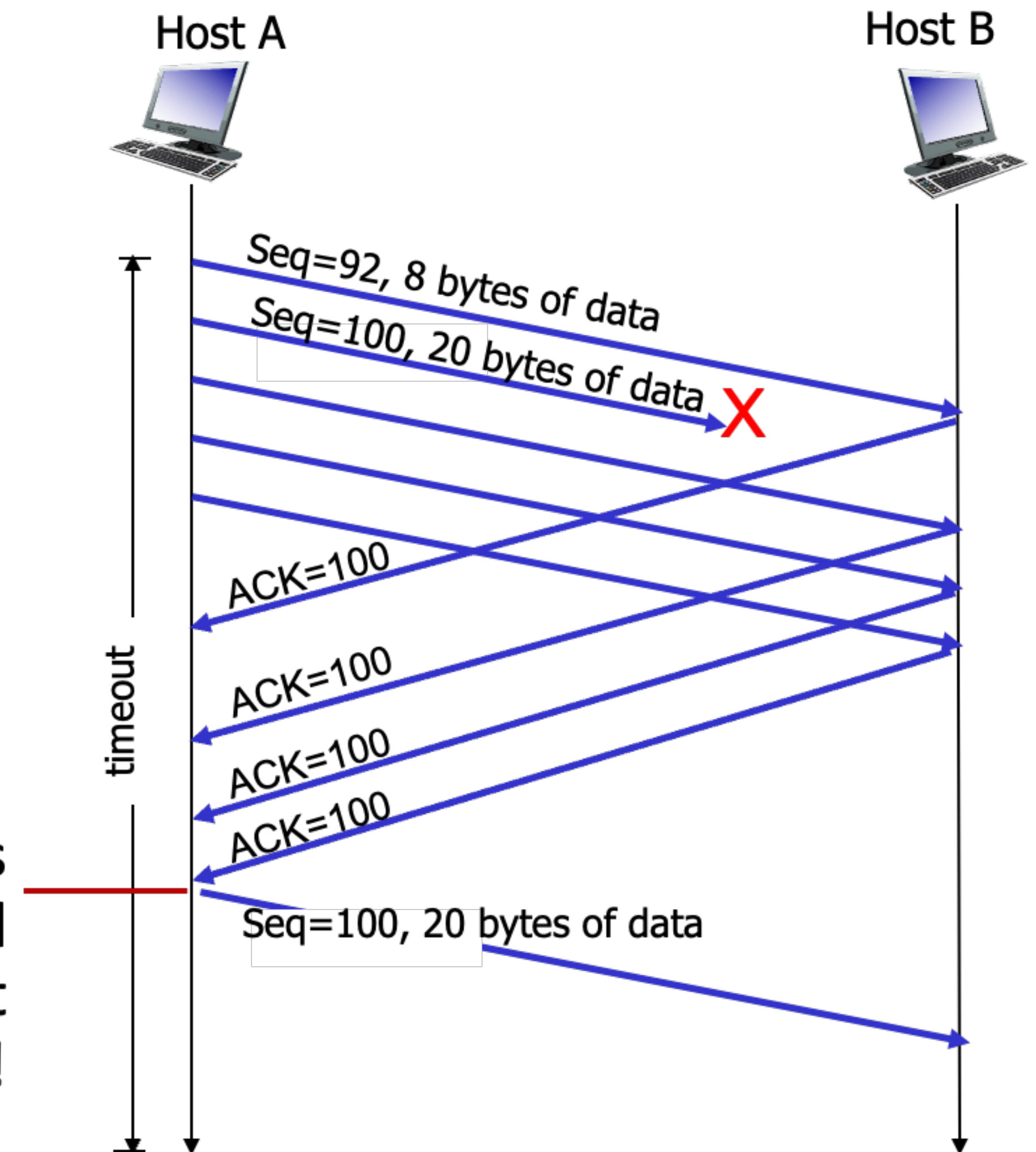
Cumulative ACK covers earlier lost ACK

TCP fast retransmit

- If sender receives 3 additional ACKs for the same data (“triple duplicate ACKs”) resend unACKed segments with smallest seq. #
- Underlying principle: it is likely that unACKed segment lost, so don't wait for timeout

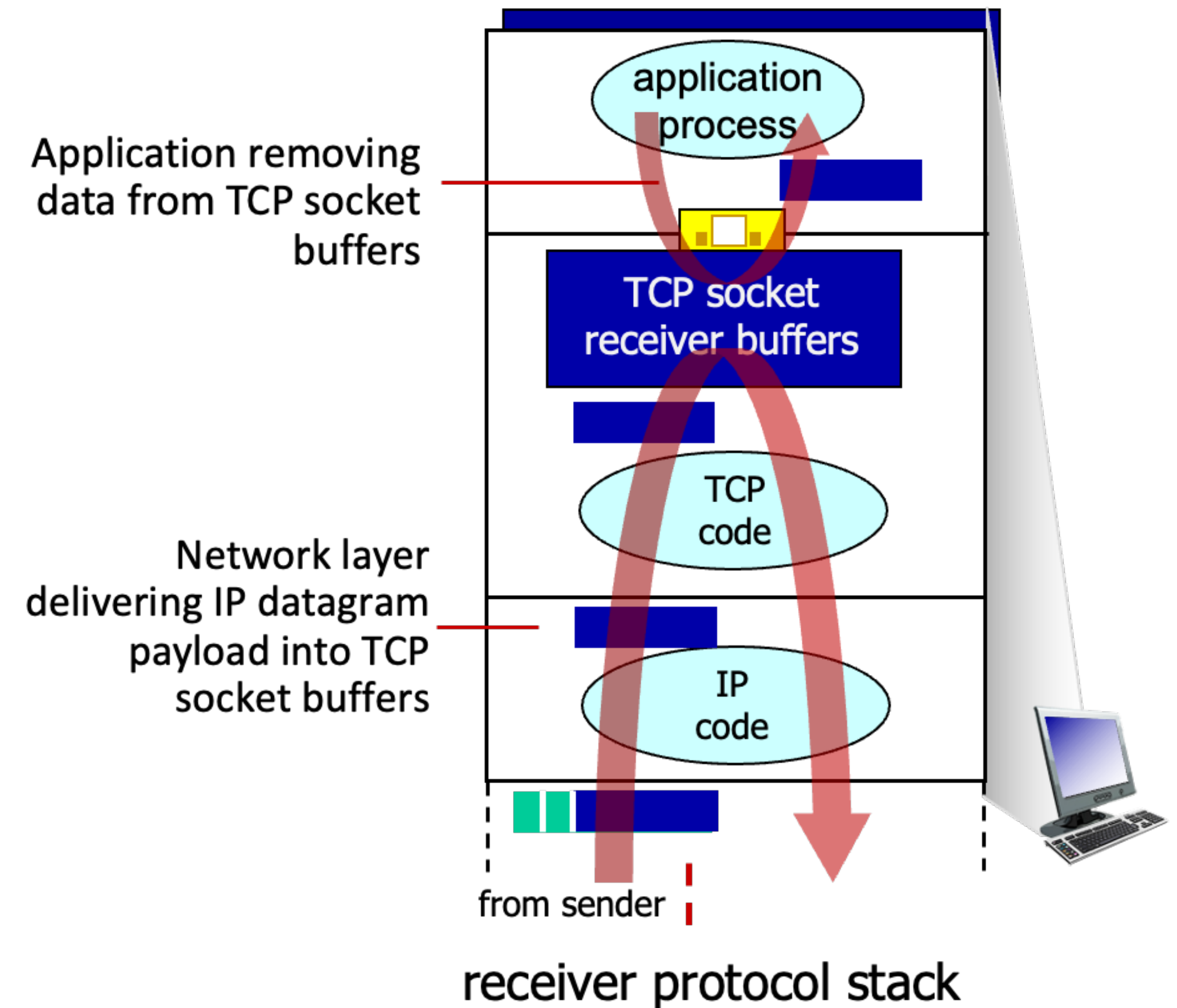


Receipt of three duplicate ACKs indicates 3 segments received after a missing segment – lost segment is likely. So retransmit!



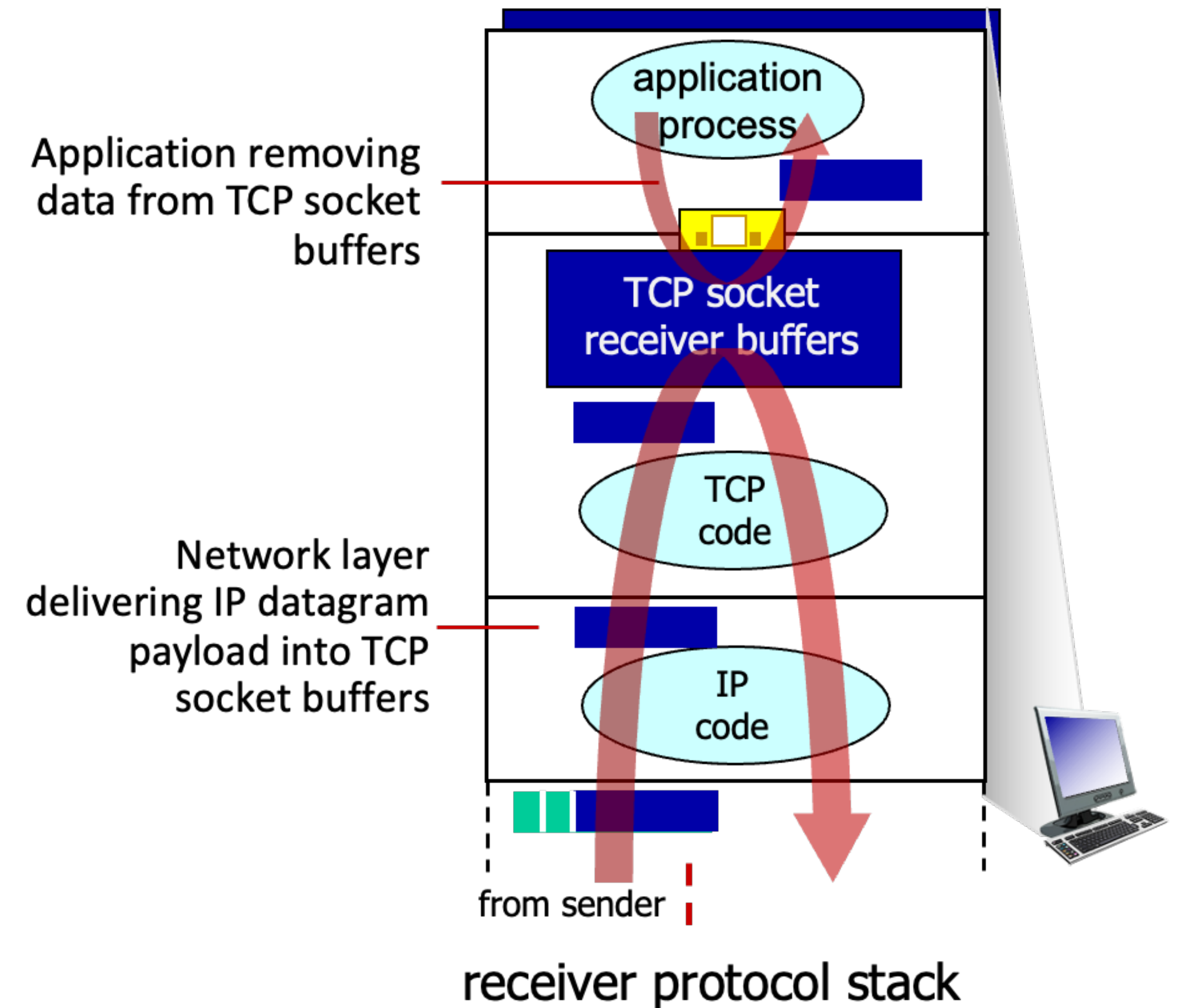
TCP flow control

- What happens if network layer delivers data faster than application layer removes data from socket buffers?



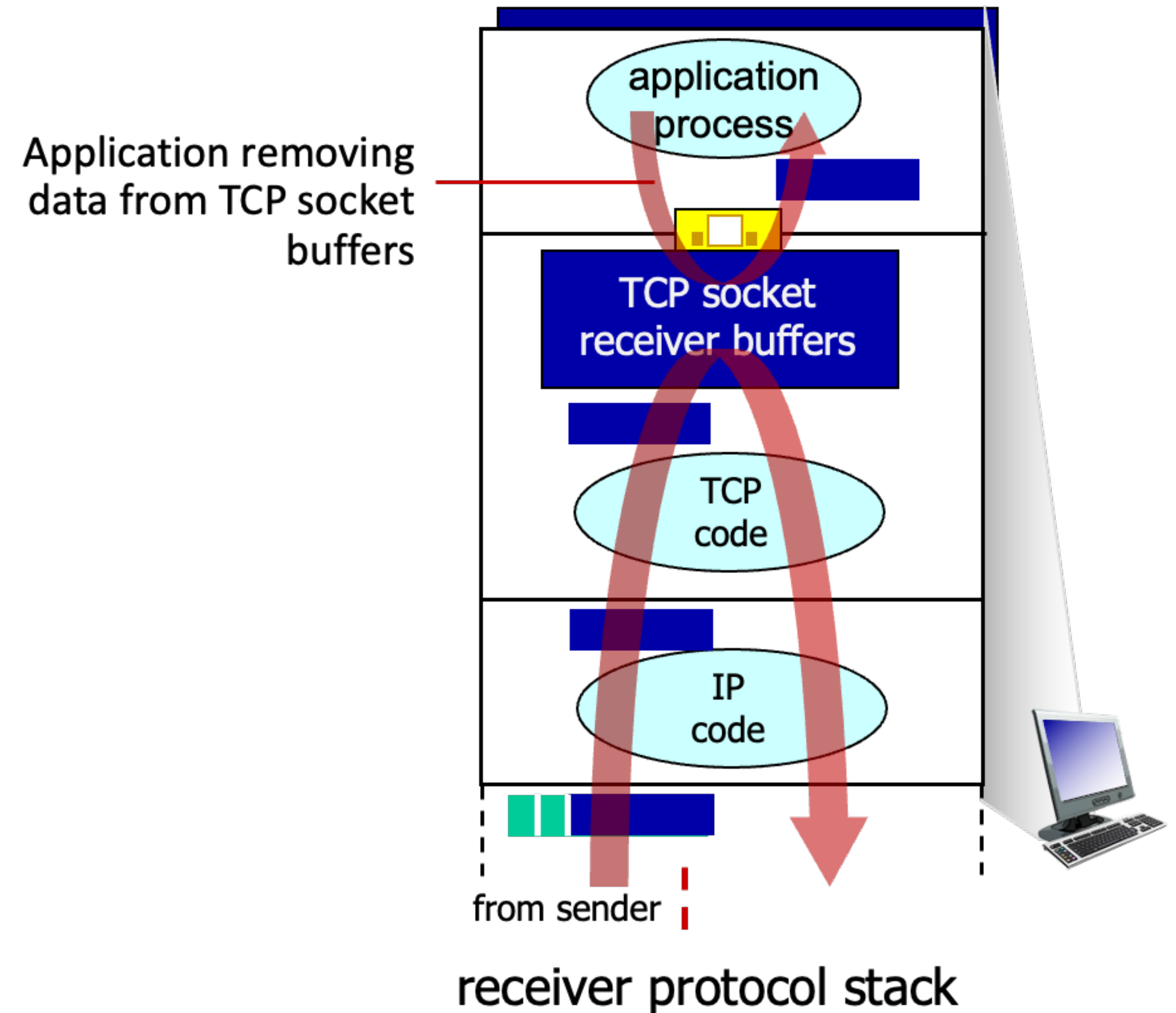
TCP flow control

- What happens if network layer delivers data faster than application layer removes data from socket buffers?



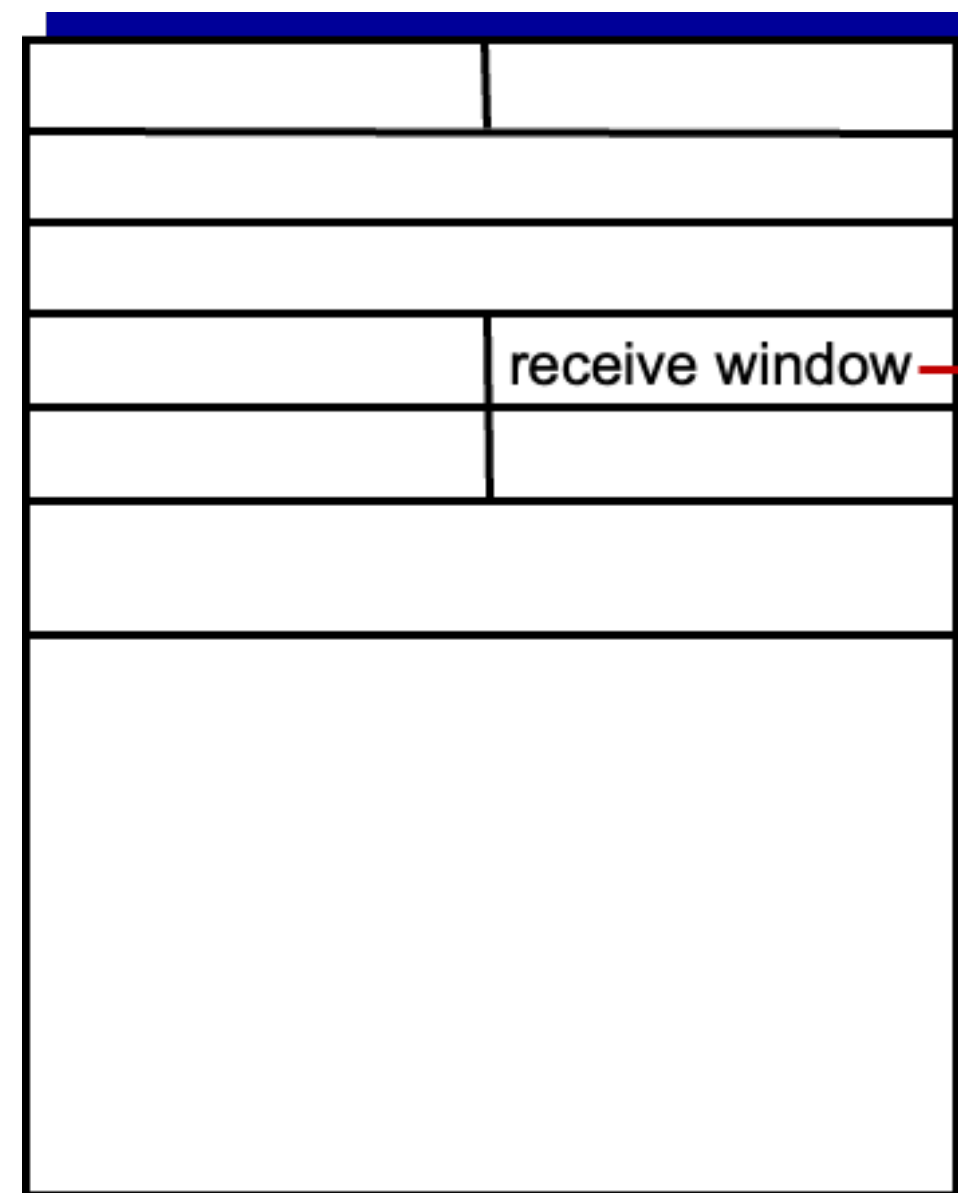
TCP flow control

- What happens if network layer delivers data faster than application layer removes data from socket buffers?
- **Flow control:** receiver controls sender, so sender won't make receiver's buffer overflowed by transmitting too much, too fast



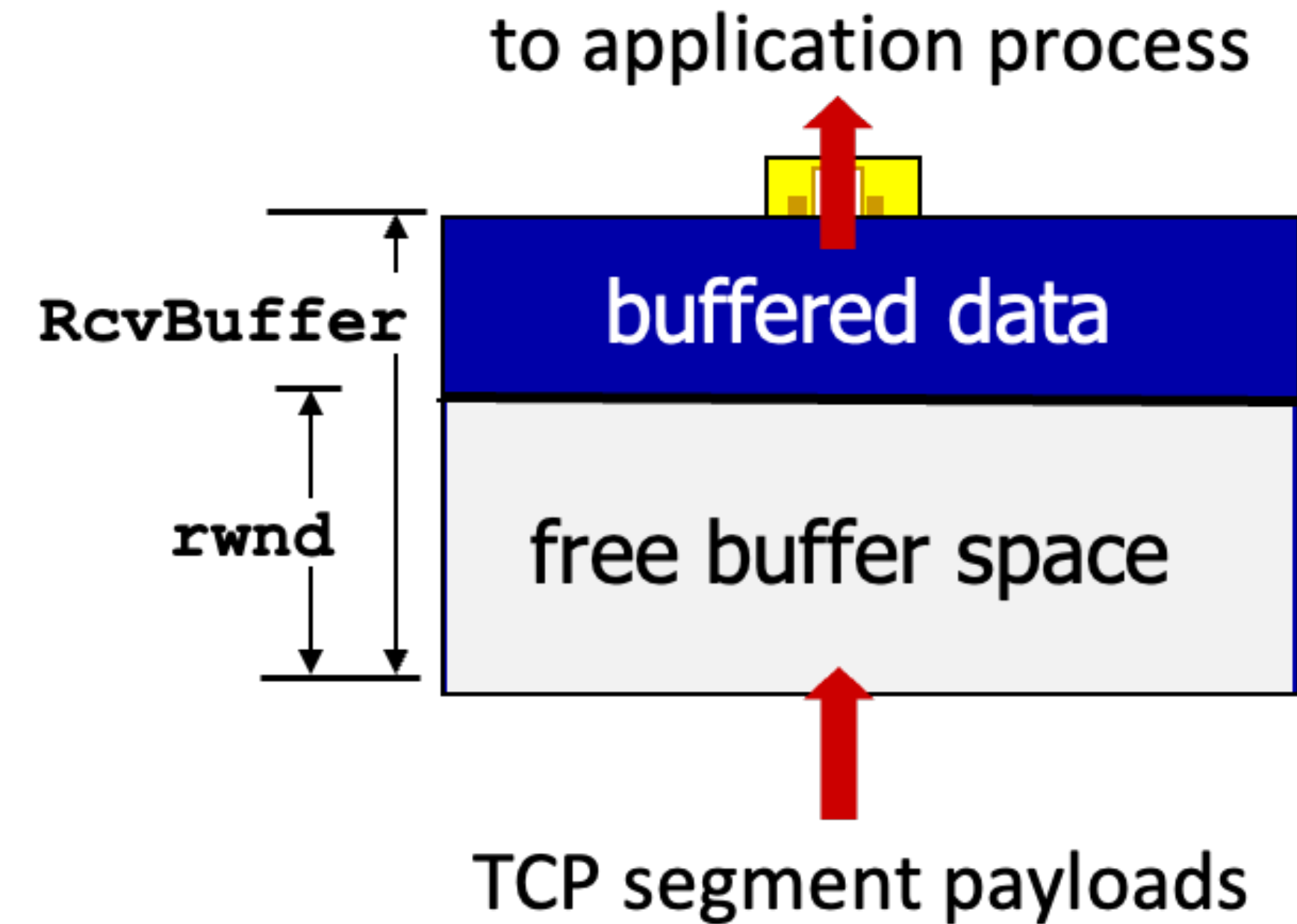
TCP flow control

- TCP receiver “advertise” free buffer space in **rwnd** field in TCP header



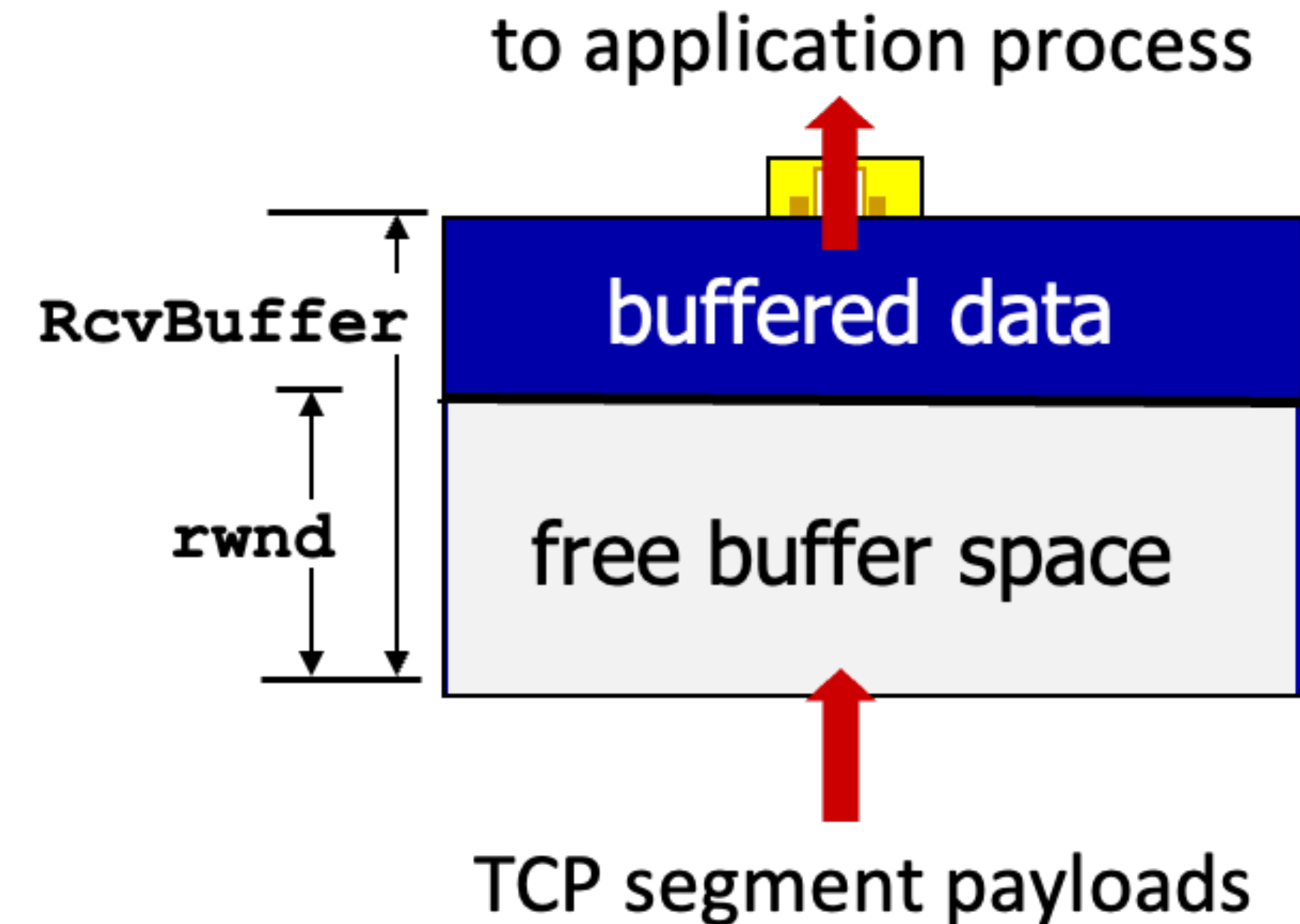
TCP segment format

flow control: # bytes
receiver willing to accept



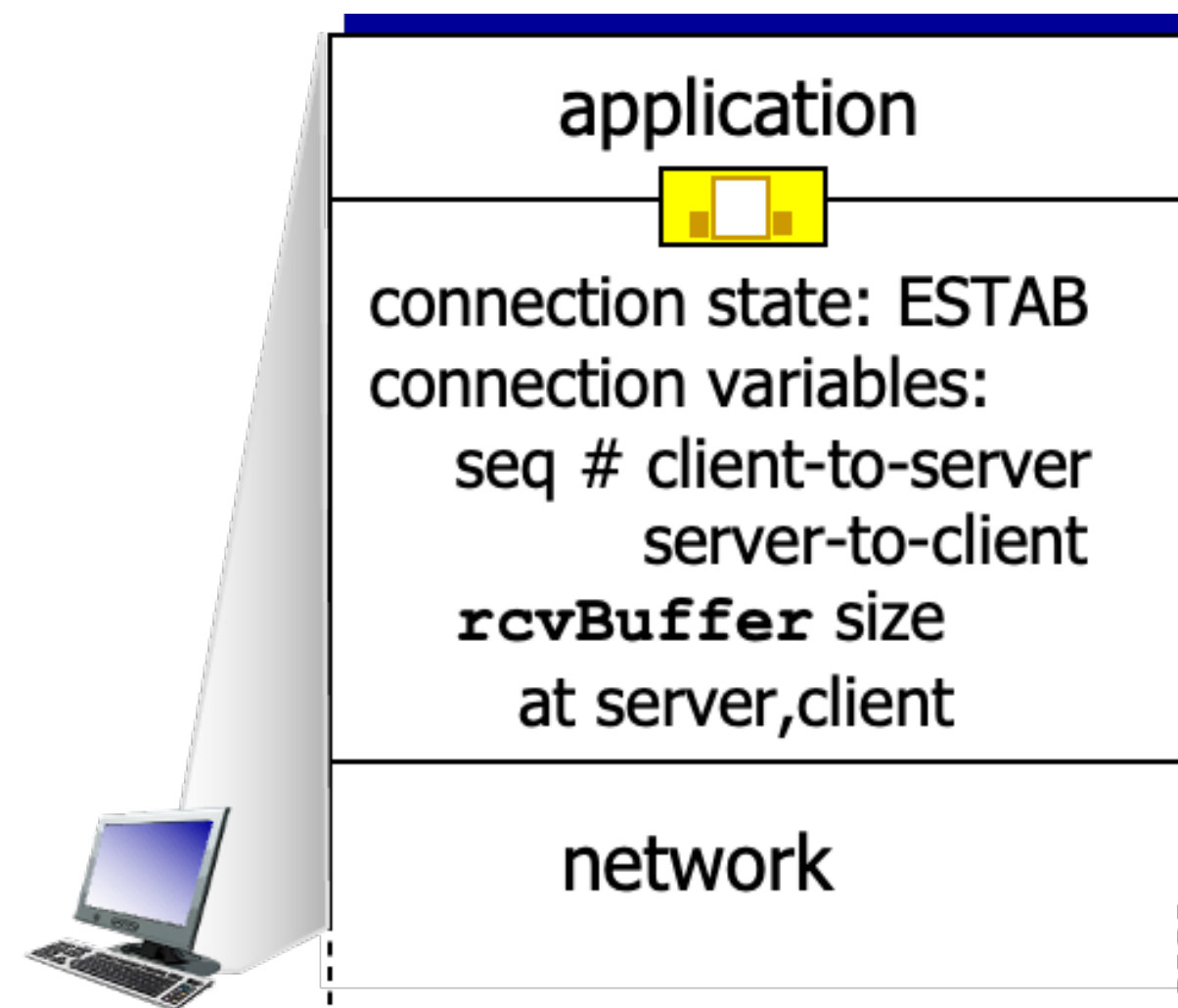
TCP flow control

- TCP receiver “advertise” free buffer space in **rwnd** field in TCP header
 - **RcvBuffer** size set via socket options (typically, default is 4096 bytes)
 - Many operating systems auto-adjust **RcvBuffer**
- Sender limits amount of unACKed (“inflight”) data to received **rwnd**, which guarantees receive buffer will not overflow

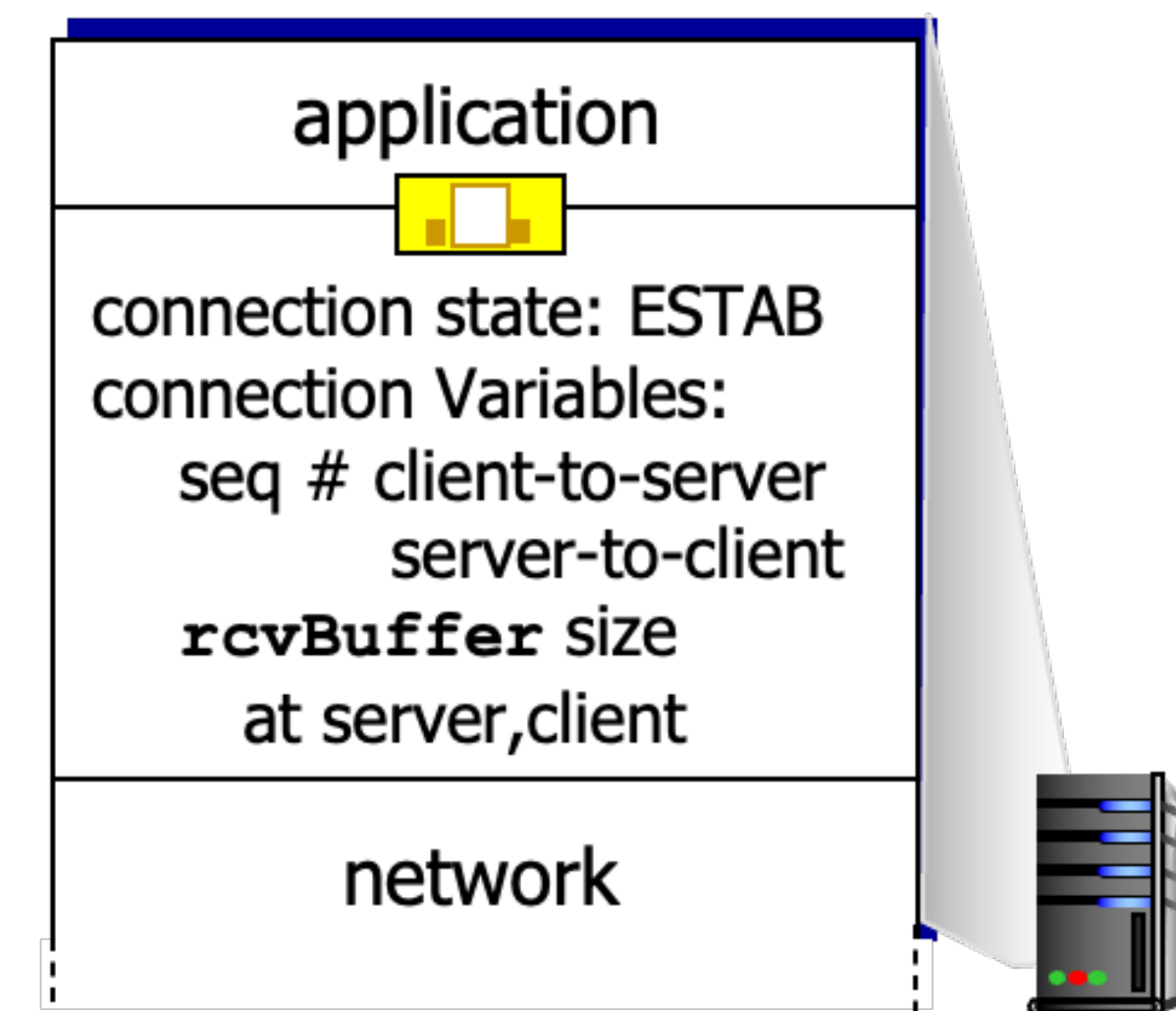


TCP connection management

- Before exchanging data, sender & receiver “handshake”
 - Agree to establish connection (each knowing the other willing to establish connection)
 - Agree on connection parameters (e.g., starting seq. #s)



```
Socket clientSocket =  
    newSocket("hostname", "port number");
```

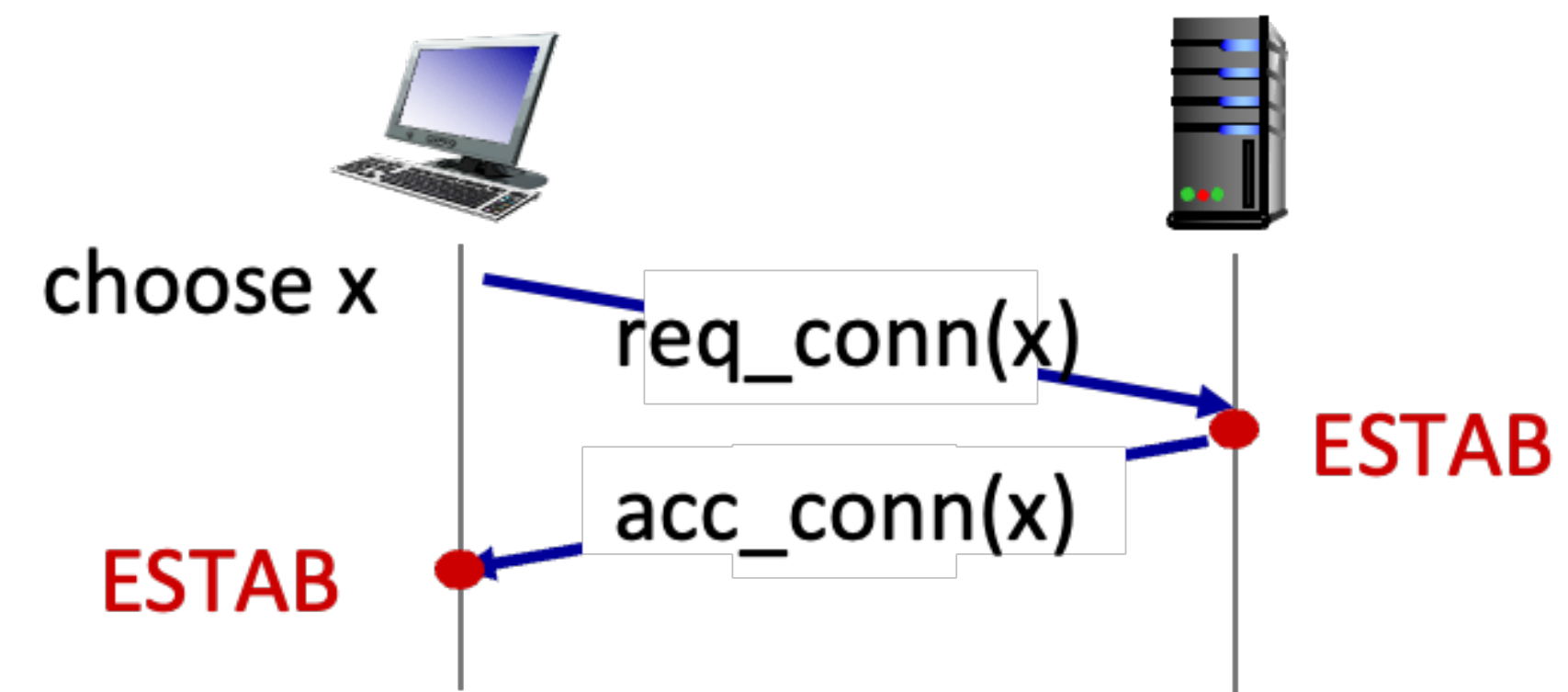
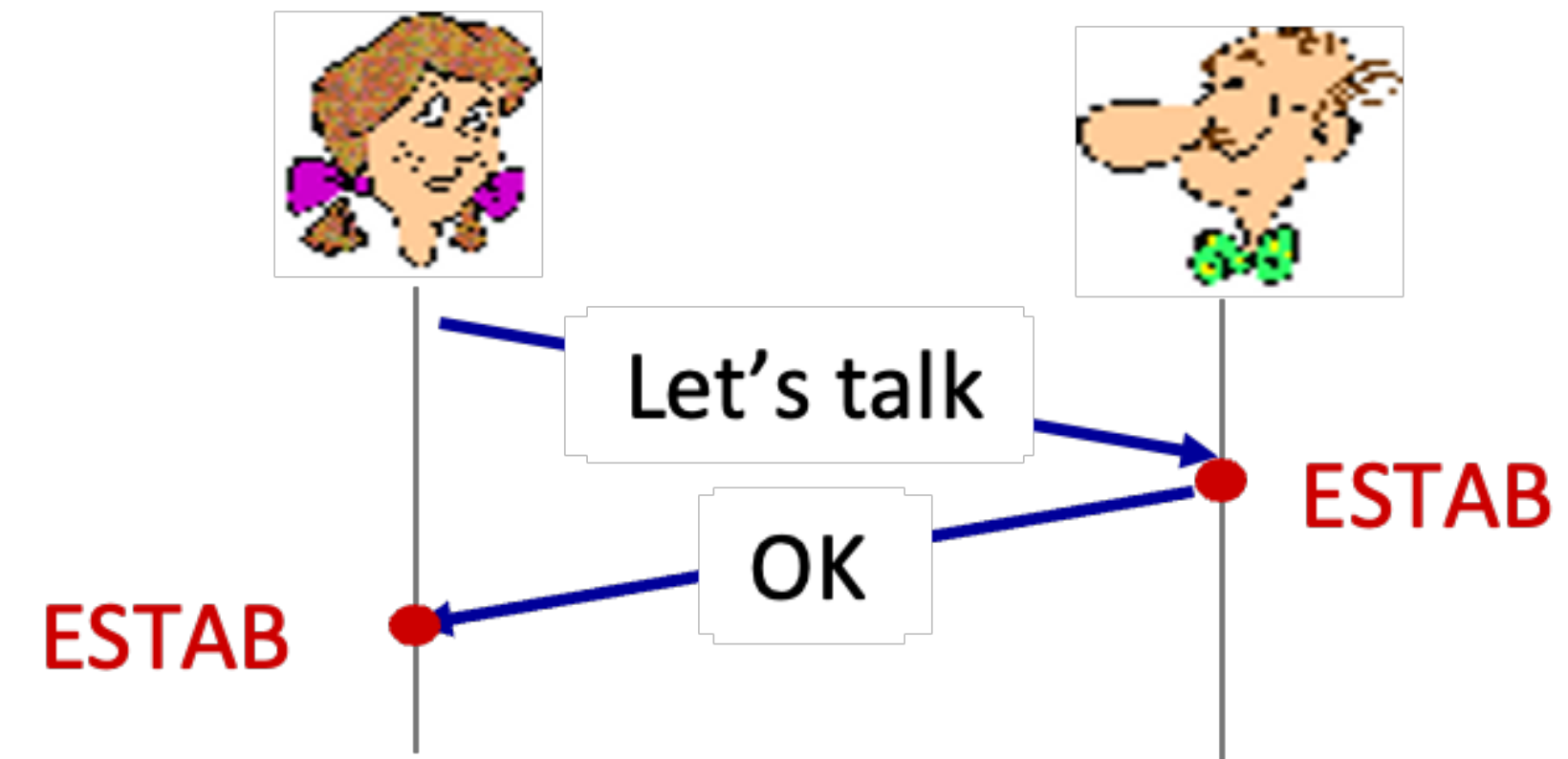


```
Socket connectionSocket =  
    welcomeSocket.accept();
```

Agreeing to establish a connection

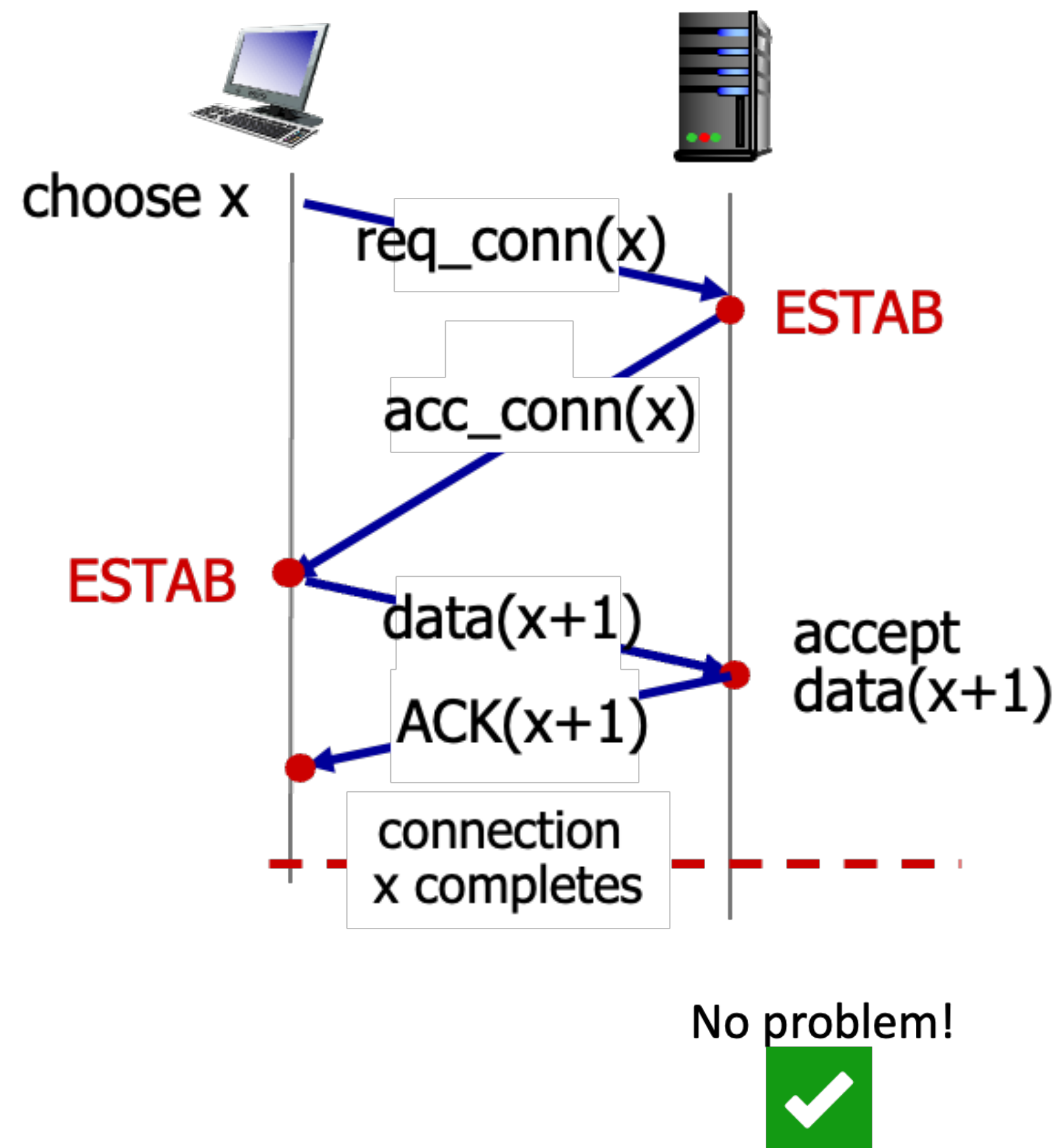
- 2-way handshake

2-way handshake:



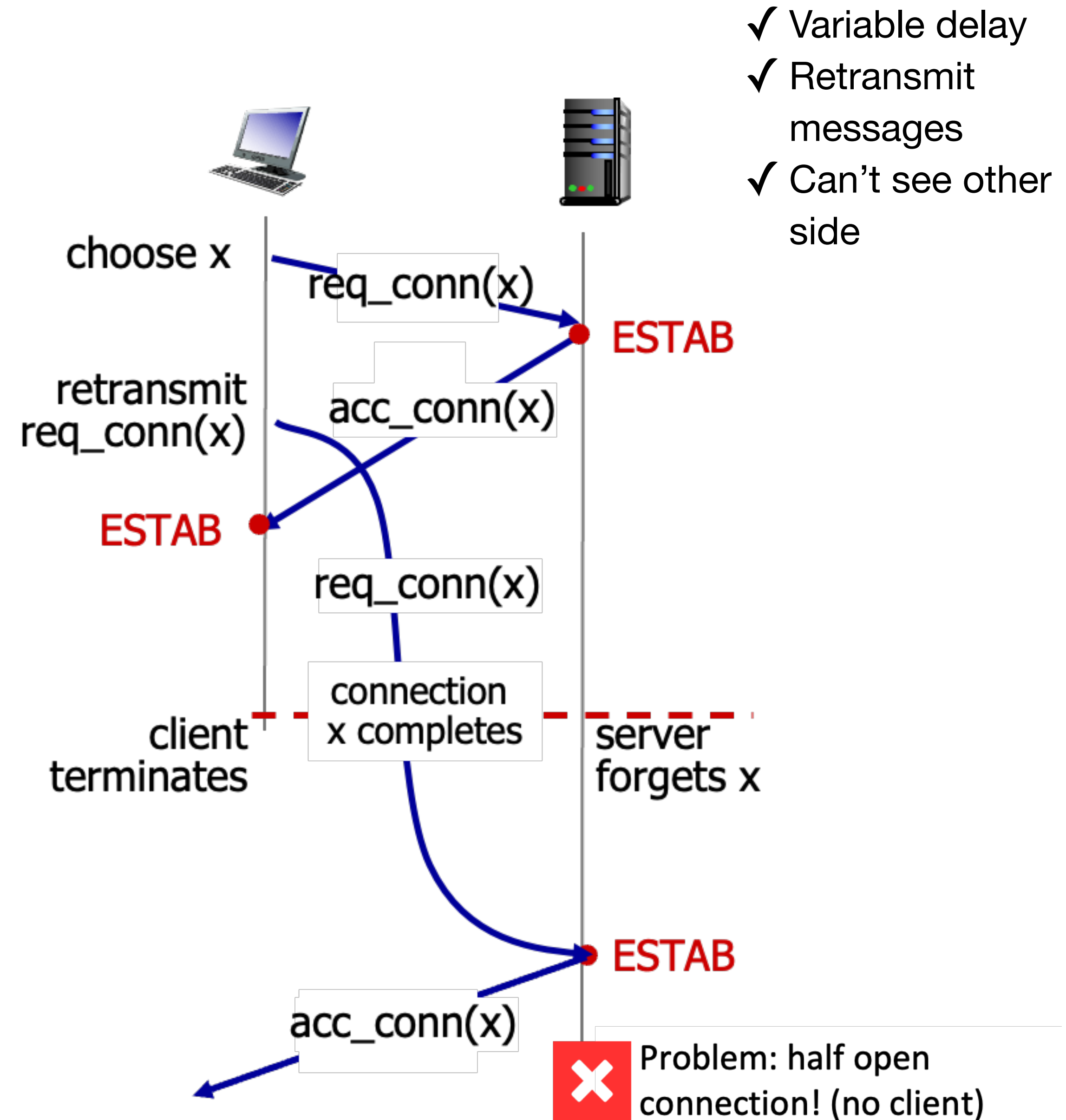
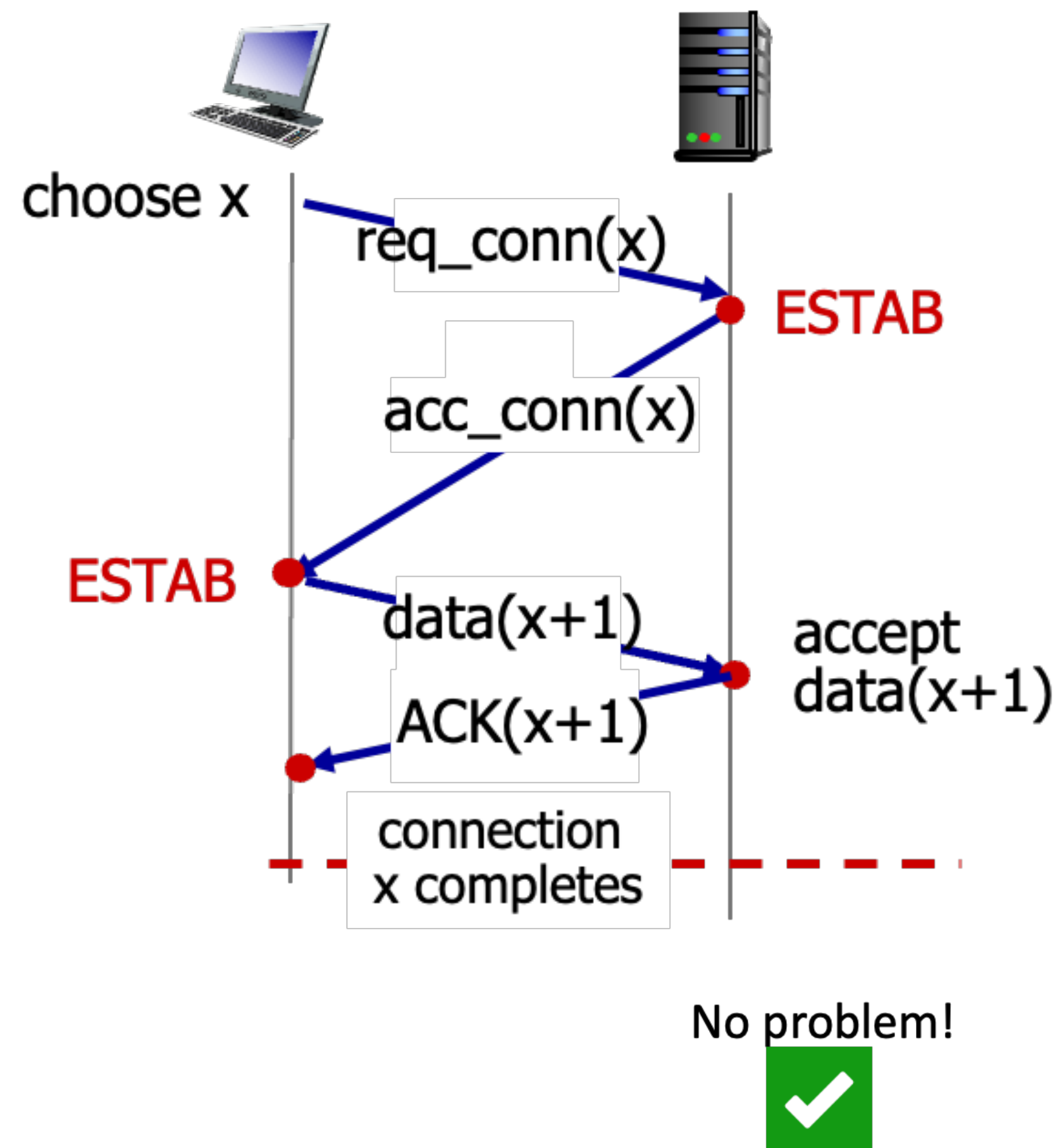
Agreeing to establish a connection

- Why not 2-way handshake in network?



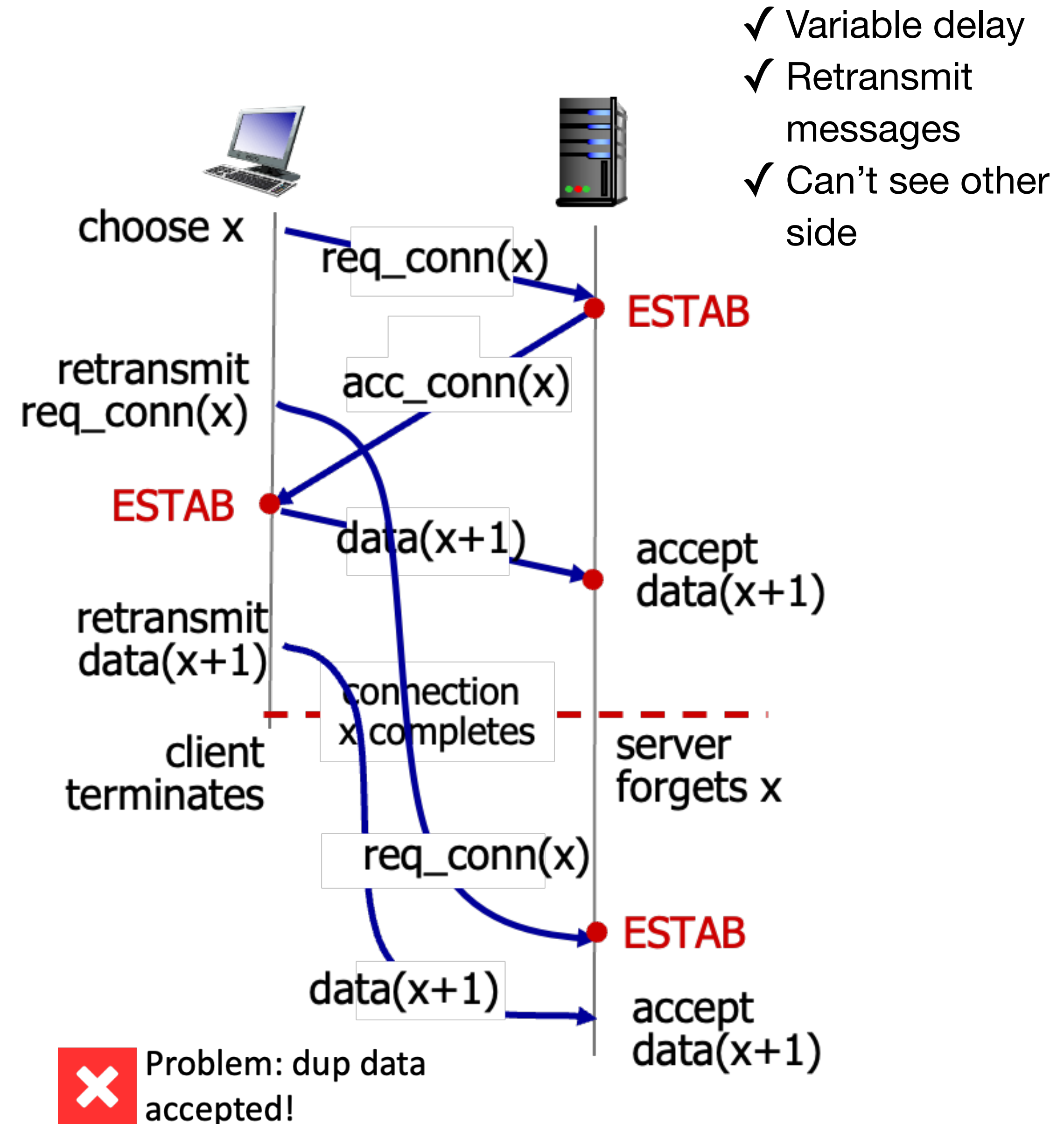
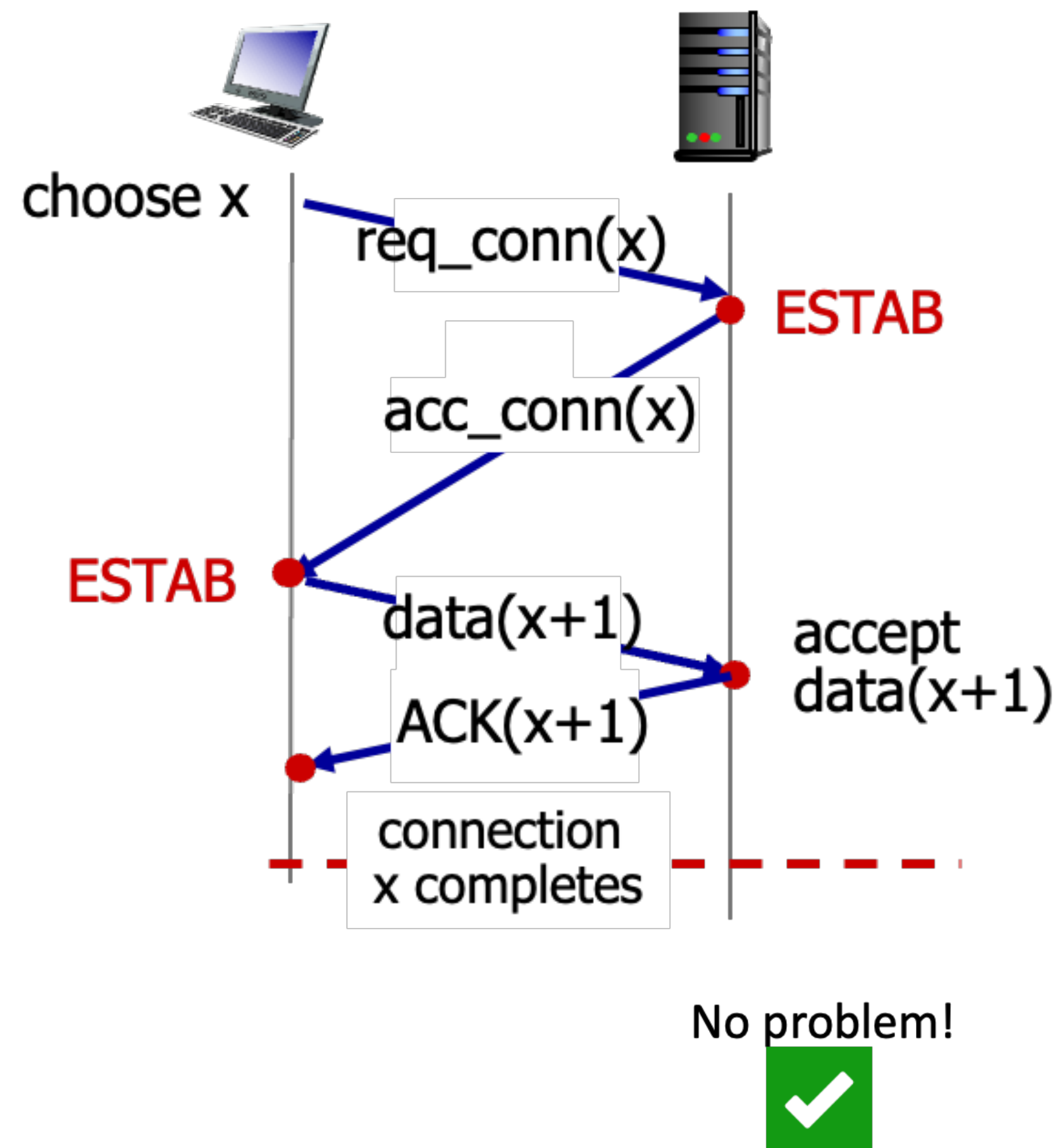
Agreeing to establish a connection

- Why not 2-way handshake in network?



Agreeing to establish a connection

- Why not 2-way handshake in network?



TCP 3-way handshake

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName, serverPort))
```

SYNSENT

ESTAB

choose init seq num, x
send TCP SYN msg

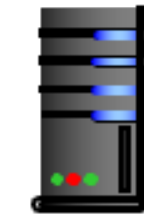
received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data



SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

ACKbit=1, ACKnum=y+1



choose init seq num, y
send TCP SYNACK
msg, acking SYN

received ACK(y)
indicates client is live

Server state

```
serverSocket = socket(AF_INET, SOCK_STREAM)  
serverSocket.bind('', serverPort)  
serverSocket.listen(1)  
connectionSocket, addr = serverSocket.accept()
```

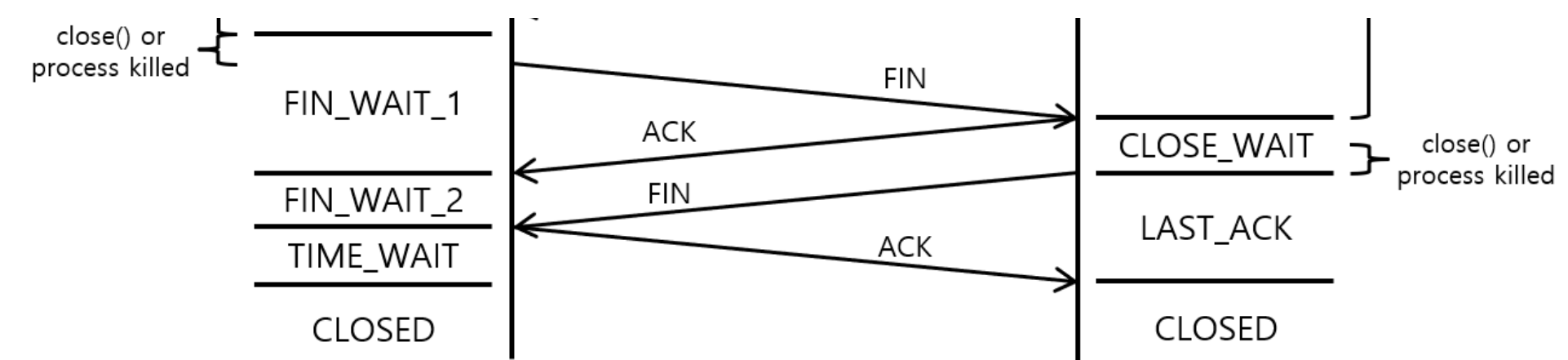
LISTEN

SYN RCVD

ESTAB

Closing a TCP connection

- Client, server each close their side of connection
 - Send TCP segment with FIN bit = 1
- Respond to received FIN with ACK
 - On receiving FIN, ACK can be combined with own FIN
- Simultaneous FIN exchanges can be handled



TCP A			TCP B		
ESTABLISHED			ESTABLISHED		
(Close)					
FIN-WAIT-1	-->	<SEQ=100><ACK=300><CTL=FIN, ACK>	-->	CLOSE-WAIT	
FIN-WAIT-2	<--	<SEQ=300><ACK=101><CTL=ACK>	<--	CLOSE-WAIT	
TIME-WAIT	<--	<SEQ=300><ACK=101><CTL=FIN, ACK>	<--	LAST-ACK	(Close)
TIME-WAIT	-->	<SEQ=101><ACK=301><CTL=ACK>	-->	CLOSED	
(2 MSL)					
CLOSED					

Questions?